

decoding the received IR signal from the NEC remote transmitter.

IR remote transmitter and receiver

In this setup, we use the NEC protocol IR remote control. Fig. 4 illustrates the NEC message frame, while Table 2 provides a list of NEC remote codes. Fig. 5 displays the IR remote and IR receiver module.

The choice of using the NEC protocol is based on the availability, small form factor, and low cost of the IR transmitter. The transmitter operates at a 38kHz carrier frequency. There are two types of IR receivers: one works within a wide input range (2.8V to 5V), while the other functions solely at 5V. For this radio, we utilise the wide voltage input type of remote receiver, which functions well at 3.3V. It has only three

pins, which can be connected as per the circuit diagram shown here. Ensure that the centre frequency of your IR receiver module is 38kHz.

Tuning the FM radio module can be achieved in two ways: direct frequency tuning or seeking available channels in both directions. We have opted for fixed frequency tuning in this radio.

The NEC message frames with timing requirements are depicted in Fig. 4. We do not utilise the 'REPEAT' feature by pressing a button for a longer duration. It is important to define the default volume in the code. You will also need to edit the local FM channel frequencies in the source code. Fig. 6 presents a code snippet, including the necessary library, while Fig. 7 shows a code snippet for setting defaults.

The variable 'Maxc' should be

set to the total number of stations listed, counting from zero. The 'Channel = 1;' line defines the default station during startup. After compilation, flash the ESP-01 using any USB-to-TTL converter or a dedicated ESP-01 programmer board with Tx and Rx lines set at a baud rate of 115200. Fig. 8 showcases the ESP-01 programmer used for flashing.

You also have the option to set and change the OLED display variants in the setup function within the code. Two varieties are available, as shown in Fig. 9. Fig. 10 shows the ESP 01 board.

TABLE 2
NEC REMOTE CONTROL CODES

S.No.	KEY	HEX
1	ON/OFF	1FE48B7
2	MODE	1FE58A7
3	MUTE	1FE7887
4	PAUSE/PLAY	1FE807F
5	PREVIOUS	1FE40BF
6	NEXT	1FEC03F
7	EQ	1FE20DF
8	VOL-	1FEA05F
9	VOL+	1FE609F
10	0	1FEE01F
11	RPT	1FE10EF
12	U/SD	1FE906F
13	1	1FE50AF
14	2	1FED827
15	3	1FEF807
16	4	1FE30CF
17	5	1FEB04F
18	6	1FE708F
19	7	1FE00FF
20	8	1FEF00F
21	9	1FE9867

```

// TX - 4.7kQ-/\- 3V3 [UNUSED]-AVAILABLE FOR USER
#include <Wire.h>
#include <radio.h>
#include <RDA5807M.h>
#include <RDSParser.h>
#include <IRremote.h>
#include "SSD1306Ascii.h"
#include "SSD1306AsciiWire.h"
#define I2C_ADDRESS 0x3C // make sure your OLED'S I2C address is correct

#define RST_PIN -1 // Define proper RST_PIN if required.

RDA5807M radio;
RDSParser rds;
SSD1306AsciiWire oled;

/// State definition for this radio implementation.
enum RADIO_STATE {
    STATE_PARSECOMMAND, //<- waiting for a new command character.
    STATE_PARSEINT,      //<- waiting for digits for the parameter.
    STATE_EXEC           //<- executing the command.
};

RADIO_STATE state; //< The state variable is used for parsing input characters.

int RECV_PIN = 3; //Connect the IR data to RX pin(I03) of ESP-01 // Rx pin // No UART possible
IRrecv irrecv(RECV_PIN);
decode_results results;

```

Fig. 6: Code snippet including library

```

int volume = 2; // start with the volume level = 2
int channellist[] = {911, 919, 927, 935, 943, 950, 983, 1001, 1013, 1029, 1030, 1040, 1064}; // Define some radio stations
int maxc = 12; // Define the number of radio stations used [0..n]
int channel = 1; // Start with a favorite station [0,1,2,3,4,...]

```

Fig. 7: Code snippet for setting defaults